

CONTAINER WITH MOST WATER (LeetCode 11)

1. PROBLEM STATEMENT

You are given an integer array `height[]` where each element represents the height of a vertical line.

Find two lines that together with the x-axis form a container that holds the maximum amount of water.

Formula: $\text{Area} = \min(\text{height}[\text{left}], \text{height}[\text{right}]) \times (\text{right} - \text{left})$

2. BRUTE FORCE APPROACH

Idea: Check every possible pair of lines and calculate the area. Choose the maximum area among all pairs.

Algorithm:

- `maxArea = 0`
- For `i = 0` to `n-1`
For `j = i+1` to `n-1`
 `height = min(arr[i], arr[j])`
 `width = j - i`
 `area = height * width`
 `maxArea = max(maxArea, area)`
- Return `maxArea`

Dry Run: `height = [1, 8, 6, 2]`

i	j	Heights (i, j)	min	width	Area
0	1	(1, 8)	1	1	1
0	2	(1, 6)	1	2	2
0	3	(1, 2)	1	3	3
1	2	(8, 6)	6	1	6
1	3	(8, 2)	2	2	4
2	3	(6, 2)	2	1	2

Maximum Area = 6

Complexity:

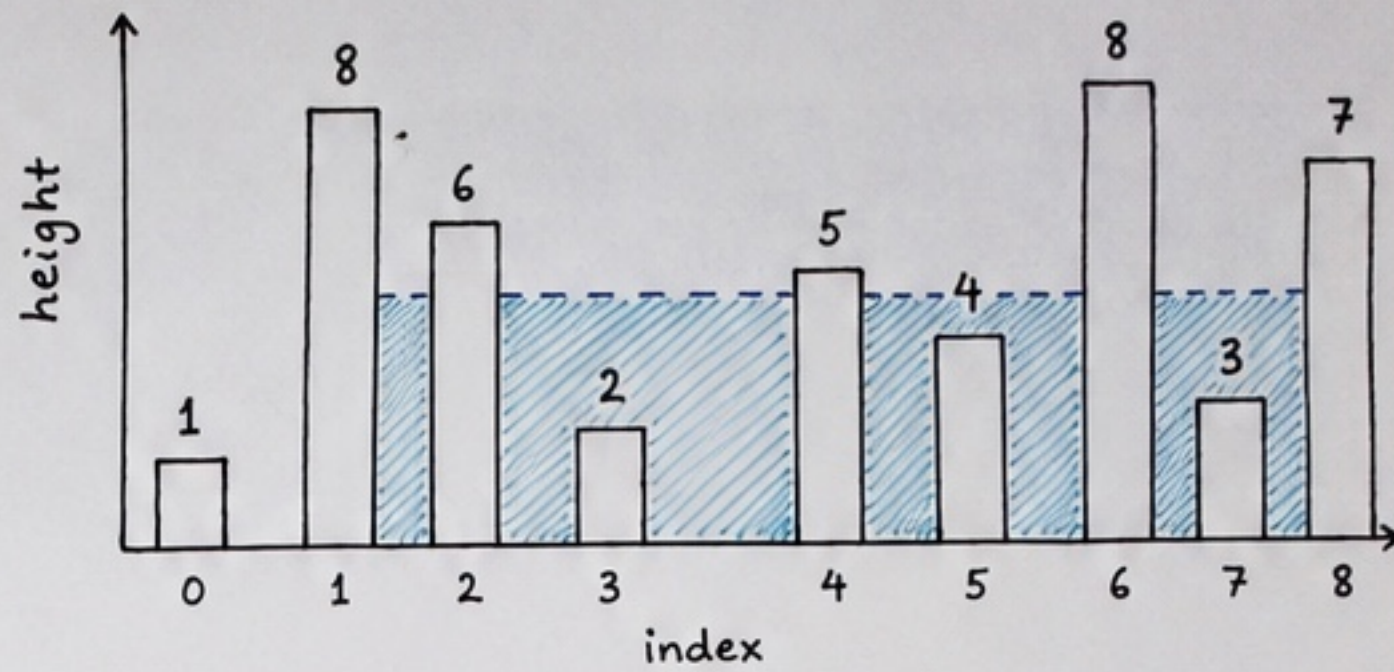
- Time : $O(n^2)$
- Space : $O(1)$

Why slow? For every element, we compare it with every other element.

$$\text{Total comparisons} = \frac{n(n-1)}{2} \approx O(n^2)$$

NOTE / TAKEAWAY

- Brute Force checks all pairs. ($O(n^2)$)
- Two Pointer approach is optimal. ($O(n)$)
- Always move the pointer with smaller height.



3. OPTIMAL APPROACH (TWO POINTERS)

Observation: Area depends on two things

$$\text{Area} = \text{Height} \times \text{Width}$$

- Width decreases whenever pointers move.
- Therefore, we should try to increase the limiting height.

Since height is determined by the smaller line, only moving the smaller line can possibly increase the area.

Key Intuition:

Suppose `left = 3`, `right = 7`
`Height = min(3, 7) = 3`
`Current area = 3 * width`

If we move the taller line (7):

- Width decreases
- Minimum height remains 3
- Area can only decrease or remain the same.

If we move the smaller line (3):

- Width decreases
- But we may find a taller line than 3.
- Minimum height may increase
- Area may increase.

Therefore always move the pointer with smaller height.

Algorithm:

- `left = 0`, `right = n-1`, `maxArea = 0`
- While (`left < right`)
 - `height = min(arr[left], arr[right])`
 - `width = right - left`
 - `area = height * width`
 - `maxArea = max(maxArea, area)`
 - If `arr[left] < arr[right]`
 `left++`
else
 `right--`
- Return `maxArea`

Dry Run: `height = [1, 8, 6, 2, 5, 4, 8, 3, 7]`

Step	Left	Right	Heights (L, R)	Width	Area	Max Area	Move
1	0	8	(1, 7)	8	8	8	Left++
2	1	8	(8, 7)	7	49	49	Right--
3	1	7	(8, 3)	6	18	49	Right--
4	1	6	(8, 8)	5	40	49	Right--
5	1	5	(8, 4)	4	32	49	Right--
6	1	4	(8, 5)	3	24	49	Right--
7	1	3	(8, 2)	2	16	49	Right--
8	1	2	(8, 6)	1	6	49	Right--

Maximum Area = 49

Complexity (Optimal):

- Time : $O(n)$
- Space : $O(1)$